MFGraphs Package Documentation

Per-module long-form reference

MFGraphs maintainers

Contents

Part I

Loader

Chapter 1

MFGraphs.wl – Top-level loader

1.1 High-level explanation

`MFGraphs.wl` is the top-level loader for the active MFGraphs package surface. Its job is not to implement mathematics directly, but to make the package usable through a single `Needs["MFGraphs‘"]` call. Concretely, it prepends the package directory to `$Path`, declares the dependency contexts, and then relies on `EndPackage[]` to expose the public symbols from the active subpackages on the Wolfram context path.

The file therefore acts as the architectural front door to the repository. It defines the load order and the public dependency graph: primitives first, then shared utilities, then the scenario kernel, then example constructors, then symbolic unknowns, then system construction, then solvers, orchestration, and graphics. In practice, this means that most user-facing workflows begin by loading `MFGraphs‘` rather than any subcontext individually.

Optional subpackages

Two files in the package directory are intentionally *not* loaded by `Needs["MFGraphs‘"]`: `Tawaf.wl` (unrolled circumambulation scenario builder) and `numericOracle.wl` (LP-relaxation oracle, the only file in the package that introduces floating-point work). They are loaded explicitly after the main package:

```
Needs["MFGraphs‘"];
Needs["Tawaf‘"];          (* makeTawafScenario, ... *)
Needs["numericOracle‘"];  (* numericOracleClassify, solveScenarioWithOracle *)
```

The high-level wrapper for the oracle path is `solveScenarioWithOracle`, which composes `addSymmetryEqualities` → `numericOracleClassify` → `addOracleEqualities` → `activeSetReduceSystem` and falls back to the unpruned solve when the oracle over-prunes.

1.2 Low-level explanation of functions and functionality

This file is intentionally thin. It does not define user-callable computational functions of its own. Instead, it contributes three operational actions that matter for package behavior.

Path bootstrap via `PrependTo[$Path, DirectoryName[$InputFileName]]`

This expression ensures that the directory containing the package files is available to the Wolfram loader. Without this step, the later `BeginPackage` declaration might fail to locate sibling module files when MFGraphs is loaded from outside the package directory.

In usage terms, users do not call this expression directly. Its purpose is entirely infrastructural: it makes the rest of the package importable.

Context export via `BeginPackage["MFGraphs`", {...}]`

This call declares the top-level `MFGraphs`` context and specifies the active dependency contexts that should be loaded with it: `primitives``, `utilities``, `scenarioTools``, `examples``, `unknownsTools``, `systemTools``, `solversTools``, `orchestrationTools``, and `graphicsTools``. The optional `Tawaf`` and `numericOracle`` contexts are intentionally absent from this list.

At a low level, this is how the package defines its public surface. Any symbol documented as public inside those dependent contexts becomes accessible after a single top-level load.

Context finalization via `EndPackage[]`

The closing `EndPackage[]` finalizes the loader. Operationally, it appends the dependency contexts to `$ContextPath`, which is why users can call names such as `makeScenario`, `makeSystem`, `solveScenario`, and `mfgFlowPlot` without qualifying them by subcontext after loading `MFGraphs``.

1.3 Usage guidance

The normal usage pattern is simply:

```
Needs["MFGraphs`"]
```

Users should regard this file as the package entrypoint rather than a functional API module. If someone is working on architecture, dependency order, or public symbol exposure, this is the correct file to inspect. If they are working on modeling, solving, or plotting logic, they should move immediately to the downstream modules that this file loads.

References

1. Source file: `MFGraphs/MFGraphs.wl`
2. Repository guidance: `CLAUDE.md`

Part II

Active package surface

Chapter 2

primitives.wl

2.1 High-level explanation

`primitives.wl` is the base layer of the active MFGraphs stack. It defines the symbolic atoms and runtime switches that all higher modules rely on. In architectural terms, this file separates the package's foundational vocabulary from all later scenario, system, solver, and graphics logic.

The file has two kinds of responsibility. First, it establishes the core symbolic heads `j`, `u`, `z`, `alpha`, and `Cost`. These are the building blocks from which unknown bundles, systems, and plotted labels are assembled. Second, it defines package-wide runtime flags and lightweight utilities for verbosity and kernel availability. That makes it the natural place for low-level conventions that must be shared uniformly across the rest of MFGraphs.

2.2 Low-level explanation of functions and functionality

j

`j` is the package's symbolic flow head. In two-argument form, `j[a,b]` denotes flow on the directed edge from `a` to `b`. In three-argument form, `j[r,i,w]` denotes the transition-flow quantity associated with arriving along edge `(r,i)` and continuing along edge `(i,w)`.

Functionally, `j` is not evaluated in this file; it is a symbolic container. Its main usage is downstream, where `unknownsTools.wl` constructs families of `j` variables and `systemTools.wl` places them into equations, inequalities, and complementarity alternatives.

u

`u[a,b]` represents the value-function variable associated with the endpoint `b` of the directed edge `(a,b)`. Like `j`, it is a symbolic head rather than a numerically evaluated function.

Its usage is primarily in Hamilton–Jacobi and switching optimality expressions. Users normally encounter `u` through systems, solver results, and plotting functions rather than by constructing it manually.

z

`z[v]` is a vertex-potential symbol. In the current active surface it plays the role of a shared primitive rather than a frequently exposed workflow object.

A practical usage note is that `z` exists to keep the symbolic vocabulary complete and consistent with broader MFGraphs modeling ideas, even if the current scenario-first workflow emphasizes `j` and `u` more heavily.

alpha

`alpha[edge]` is the congestion exponent associated with an edge. This module gives it the default rule `alpha[_] := 1`, which means the package assumes critical congestion unless edge-specific information overrides that default elsewhere.

In usage terms, this default is extremely important because several active solvers only support systems with `Alpha == 1` on every edge. The primitive definition here therefore anchors the solver regime expected by the current active package phase.

Cost

`Cost[m, edge]` is the congestion cost function. The implementation in this module is `m^alpha[edge]`, so the edge-specific exponent can alter how congestion cost scales with flow magnitude.

Users normally do not call `Cost` directly unless they are extending solver or system logic. Instead, it serves as the canonical primitive that higher-level files reference when turning symbolic flow state into cost expressions.

$MFGraphsVerbose

`$MFGraphsVerbose` is the global verbosity flag. Its default value is `False`. When it is set to `True`, progress and timing messages become visible through the package's print helpers.

The intended usage pattern is operational rather than mathematical:

```
$MFGraphsVerbose = True;
```

This is useful when profiling, debugging, or running scripts that benefit from progress visibility.

$MFGraphsParallelThreshold

`$MFGraphsParallelThreshold` determines how large a list must be before the package considers using parallel iteration. Its default value is `6`.

In practice, this variable exposes a performance policy. Small jobs should avoid subkernel startup overhead; larger ones may benefit from parallelism. Setting the threshold to `Infinity` disables all such parallel switching.

$MFGraphsParallelReady

`$MFGraphsParallelReady` records whether package initialization has been propagated to subkernels. The file initializes it to `False`.

This variable is primarily for workflow control in advanced or scripted environments. Typical users can ignore it unless they are explicitly managing Wolfram subkernels.

mfgPrint

`mfgPrint[args___]` conditionally calls `Print` when `$MFGraphsVerbose` is `True`. Its purpose is to centralize debugging and progress output without scattering conditionals across the codebase.

Usage is straightforward: internal code can emit a message through `mfgPrint` and know that the runtime flag controls visibility.

mfgPrintTemporary

`mfgPrintTemporary[args___]` behaves like `mfgPrint`, but uses `PrintTemporary` instead. This is suited to transient progress display rather than persistent logging.

From a low-level standpoint, it is useful in notebooks and interactive scripts where temporary status output is preferable to permanent lines in the output stream.

ensureParallelKernels

`ensureParallelKernels[]` launches Wolfram subkernels when none are running. The implementation is deliberately minimal: it checks `$KernelCount` and calls `LaunchKernels[]` only if needed.

Usage is operational. This helper is appropriate when code is about to enter a potentially parallel phase and wants a safe one-line way to ensure subkernels exist first.

2.3 Usage guidance

This module is rarely the place where an end user starts working. Instead, it provides the symbolic and runtime substrate that higher-level workflows assume is present. Advanced users extending MFGraphs should understand it well because it defines the vocabulary that recurs everywhere else in the system.

References

1. Source file: `MFGraphs/primitives.wl`
2. Loader context: `MFGraphs/MFGraphs.wl`

Chapter 3

utilities.wl

3.1 High-level explanation

`utilities.wl` is the shared-helpers layer of MFGraphs. It sits between `primitives.wl` (which defines the core symbolic heads and runtime flags) and every downstream module that needs typed-object infrastructure, rule manipulation, exact boundary handling, or critical-congestion guards. The file is loaded by `MFGraphs.wl` as part of the active package surface; users who load `MFGraphs‘` get its public symbols on the unqualified context path.

The module exists so that the same handful of conventions — how typed wrappers are recognized, how rule lists are merged and normalized, how numeric boundary data is rationalized into exact form, and how solvers refuse non-critical systems — can be reused without each subpackage reimplementing them.

3.2 Low-level explanation of functions and functionality

`mfgTypedQ` and `mfgData`

`mfgTypedQ[obj, head]` returns `True` when `obj` has the form `head[_Association]`. It is the generic predicate behind `scenarioQ`, `symbolicUnknownsQ`, `mfgSystemQ`, and similar typed-object checks. `mfgData[obj]` returns the wrapped association; `mfgData[obj, key]` returns the value for a key or `Missing["KeyAbsent", key]`.

`mergeRules`, `normalizeRules`, and `extractRules`

`mergeRules[oldRules, newRules]` joins rule lists while keeping the latest rule for each left-hand side. `normalizeRules[rules]` pushes the full rule set through the right-hand sides so accumulated substitutions become internally consistent. `extractRules[sol]` pulls replacement rules out of either a flat rule list or an association-shaped solver result. These three helpers are the backbone of multi-stage symbolic elimination in the solver layer.

`roundValues`

`roundValues[x]` rounds numeric content to a standard precision (10^{-10}) with overloads for numbers, rules, lists, and associations. It is used to stabilize reporting output without altering the exact symbolic solutions.

`exactBoundaryValue` and `exactBoundaryValues`

`exactBoundaryValue[val]` converts a numeric real boundary value to an exact rational via `Rationalize[val, 0]` and returns a `Failure` object for nonreal or nonnumeric inputs. `exactBoundaryValues[vals]` broadcasts that conversion over a list and short-circuits on the first failure. These functions guarantee that the symbolic system pipeline sees only exact boundary data.

`criticalCongestionSystemQ`

`criticalCongestionSystemQ[sys]` returns `True` when every edge in `sys` has `Alpha == 1`. It is the predicate used by the critical-congestion solver family before doing any symbolic work.

`withCriticalCongestionSolver` and `withCriticalCongestionGuard`

`withCriticalCongestionSolver[sys, solverName, bodyFunc]` wraps the shared solver scaffolding: it validates that `sys` is a critical congestion system, builds the standard solver inputs, invokes `bodyFunc` on those inputs, and then attaches the accumulated rules to the result. `withCriticalCongestionGuard[s solverName, body]` is the lighter form: it evaluates `body` only on critical systems and otherwise emits the `MFGraphs::noncritical` message and returns a `Failure`. Together they ensure that every critical-congestion solver entrypoint has the same precondition handling without duplicated boilerplate.

3.3 Usage guidance

End users rarely call `utilities.wl` symbols directly; they are infrastructure for the scenario, system, and solver layers. The two exceptions are `mergeRules` and `normalizeRules`, which are useful when post-processing solver output by hand, and `criticalCongestionSystemQ`, which is the cleanest way to ask whether a particular system is in the regime the active solvers support.

References

1. Source file: `MFGraphs/utilities.wl`
2. Upstream dependency: `primitives.wl`
3. Downstream consumers: `scenarioTools.wl`, `systemTools.wl`, `solversTools.wl`

Chapter 4

scenarioTools.wl

4.1 High-level explanation

`scenarioTools.wl` is the typed scenario kernel of the active MFGraphs architecture. It converts raw network-and-parameter input into a validated, completed, and cached `scenario[<|...|>]` object. This is the module that establishes the contract for what a scenario is, what data it must contain, how topology is derived, and which defaults are filled automatically.

Architecturally, this file is where MFGraphs stops being a loose set of associations and becomes a disciplined object pipeline. It performs model normalization, boundary validation, switching-cost normalization, Hamiltonian normalization, topology construction, and default completion. Downstream modules therefore assume that a scenario already contains a consistent `"Model"`, a normalized `"Hamiltonian"`, and cached `"Topology"` data.

4.2 Scenario schema

A completed `scenario[...]` wraps an association with the following top-level keys.

Key	Contents
`"Model"`	The user-supplied network description: `"Vertices"`, `"Adjacency"` (or `"Graph"`), `"Entries"` (vertex/mass pairs), `"Exits"` (vertex/terminal-cost pairs), `"Switching"` (cost rows or association keyed by transition triples).
`"Hamiltonian"`	Normalized Hamiltonian parameters: `"Alpha"`, `"V"`, `"G"`, plus the edge-keyed override maps `"EdgeAlpha"`, `"EdgeV"`, `"EdgeG"`.
`"Topology"`	Cached auxiliary-graph data produced by `buildAuxiliaryTopology`: the undirected base graph, auxiliary entry/exit vertices, augmented graph, directed auxiliary pairs, and admissible transition triples.
`"Identity"`	Optional identifying metadata (name, source).
`"Benchmark"`	Optional benchmark-related fields (filled with defaults by `completeScenario`).

The "Topology" block is computed once inside makeScenario and is the reason every downstream stage runs in time proportional to its own work rather than re-walking the graph.

4.3 Low-level explanation of functions and functionality

scenario

scenario[assoc] is the typed wrapper head for scenario objects. It is intentionally simple: its meaning comes from the convention that the wrapped value is an association with keys such as "Model", "Topology", "Hamiltonian", "Identity", and "Benchmark".

Users normally do not construct scenario objects by hand. The supported path is makeScenario, which guarantees validation and completion.

scenarioQ

scenarioQ[x] returns True exactly when x matches the typed scenario wrapper pattern scenario[_Association]. It is the basic predicate used throughout the package to guard scenario-only entrypoints.

The main usage is defensive programming. Any function that expects a completed typed scenario can rely on scenarioQ in its left-hand side pattern or internal validation.

makeScenario

makeScenario[assoc] is the module's main constructor. It takes a raw association, requires a "Model" block, normalizes graph-derived data when possible, validates structural and numeric conditions, normalizes Hamiltonian specification, builds auxiliary topology, and then calls completeScenario to fill derived defaults.

This function is the recommended way to enter the active MFGraphs pipeline. Typical usage looks like:

```
s = makeScenario[<|
  "Model" -> <|
    "Vertices" -> {1,2,3},
    "Adjacency" -> {{0,1,0},{0,0,1},{0,0,0}},
    "Entries" -> {{1,10}},
    "Exits" -> {{3,0}},
    "Switching" -> {}
  |>
|>];
```

Low-level responsibilities include integer-vertex enforcement, boundary-value numeric checks, boundary-vertex membership checks, disjoint entry/exit checks, switching-cost validation, and topology construction.

validateScenario

validateScenario[s] verifies the minimum structural contract of a scenario. It checks that the input is a typed scenario, that the top-level "Model" key exists, that the model is an association, and that the required model keys are present.

Its purpose is structural validation rather than full semantic completion. This makes it useful as an early guard or a reusable substep inside makeScenario.

completeScenario

`completeScenario[s]` fills in derived scenario fields and default values. In particular, it ensures benchmark defaults are present and, when topology is available, completes the switching-cost association over the entire auxiliary transition set by filling missing costs with zero.

A subtle but important behavior is that if topology is absent, `completeScenario` warns and attempts to rebuild topology from the model before completing switching costs. The function therefore serves as a robustness layer, not only a finalizer.

scenarioData

`scenarioData[s]` returns the underlying association, while `scenarioData[s,key]` performs key lookup with `Missing["KeyAbsent", key]` fallback.

This accessor is the standard low-level read interface for every downstream module. In usage terms, it is the correct way to inspect scenario contents without depending on the raw wrapper representation.

buildAuxiliaryTopology

`buildAuxiliaryTopology[model]` constructs the cached topology block used everywhere downstream. It creates the undirected graph used as the base network, introduces synthetic auxiliary entry and exit vertices, constructs the augmented graph, derives directed pair lists, and computes admissible transition triples.

This function is performance-critical because topology construction is substantially more expensive than a simple key lookup. The module's design therefore computes it once and stores it inside the completed scenario.

deriveAuxPairs

`deriveAuxPairs[topology]` returns the directed auxiliary-pair list used to build two-argument flow unknowns and related structural data. It combines entry pairs, exit pairs, and both orientations of the base graph edges.

Its main usage is in `unknownsTools.wl` and other downstream places that need a canonical ordered pair surface.

buildAuxTriples

`buildAuxTriples[auxGraph]` returns admissible transition triples $\{v_{\text{in}}, v_{\text{mid}}, v_{\text{out}}\}$ derived from an auxiliary graph. These triples are the basis of transition-flow variables and switching-cost logic.

At a low level, this function converts the graph to directed edge pairs, adds reverse directions, and delegates to the lower-level triple builder implementation.

scenarioFailure

`scenarioFailure` is the private helper used to standardize validation failures. It returns `Failure["ScenarioValida ...]` with a message and optional missing-key list.

Its value is consistency. All validation failures produced by this module therefore share a predictable shape.

hamiltonianGTermQ

`hamiltonianGTermQ[value]` recognizes valid forms for Hamiltonian `G`: either numeric values or pure functions. It is part of the Hamiltonian-validation path.

Users do not normally call it directly, but it clarifies which forms of `"G"` the current scenario schema accepts.

buildAdjacencyFromGraph

`buildAdjacencyFromGraph[graph, vertices]` derives an adjacency matrix from a Wolfram `Graph`, ordered according to the requested vertex list. If unknown vertices are requested, it returns `$Failed`.

This helper exists to make `makeScenario` flexible when input provides a graph object but omits explicit adjacency data.

normalizeScenarioModel

`normalizeScenarioModel[model]` fills missing `"Vertices"` and `"Adjacency"` from an embedded graph when possible, and also stores a normalized undirected graph representation.

The low-level purpose is to make graph-based and adjacency-based scenario construction converge to one canonical internal shape.

boundaryValuesNumericQ

`boundaryValuesNumericQ[model]` checks whether entry and exit boundary values are present in pair form and numerically valued. This is stricter than merely checking list structure.

It is used inside `makeScenario` to ensure that downstream symbolic-system construction does not receive malformed boundary conditions.

disjointBoundaryVerticesQ

`disjointBoundaryVerticesQ[model]` returns `True` when the entry-vertex set and the exit-vertex set do not overlap. The active scenario kernel rejects models where a single vertex is simultaneously declared an entry and an exit, since the auxiliary-topology construction would then attach two incompatible boundary fixtures to the same node.

boundaryVerticesPresentQ

`boundaryVerticesPresentQ[model]` returns `True` when every entry and exit vertex named in the model also appears in the `"Vertices"` list. It catches the common authoring mistake of declaring boundary data on a vertex that was never added to the graph.

switchingCostsNumericQ

`switchingCostsNumericQ[model]` validates the `"Switching"` field, supporting either list-of-rows or association form. Accepted values are numeric costs and `Infinity` as an explicit blocked-transition sentinel.

The main usage note is that malformed switching entries are rejected before topology completion and before solver construction.

`normalizeSwitchingCosts`

`normalizeSwitchingCosts[sc]` converts list-style switching rows into an association keyed by triples and leaves associations unchanged. This gives the rest of the pipeline a single canonical representation.

`integerVertexLabelsQ`

`integerVertexLabelsQ[model]` enforces the current active policy that scenario vertices must be integer-labeled. This keeps the topology and example pipeline simple and predictable.

`modelDirectedEdgePairs`

`modelDirectedEdgePairs[model]` extracts directed edge pairs from the model's vertices and adjacency matrix. It is used chiefly by Hamiltonian edge-parameter validation.

`normalizeHamiltonianSpec`

`normalizeHamiltonianSpec[spec, model]` expands default Hamiltonian fields and validates both default values and edge-specific overrides. It ensures that `"Alpha"`, `"V"`, and `"G"` all have acceptable default forms, and that edge override associations only refer to valid edges.

Usage-wise, this function is why end users can provide either a minimal Hamiltonian block or a partially specified one and still obtain a fully normalized scenario.

`buildAuxTriplesImpl`

`buildAuxTriplesImpl[directedEdges]` performs the actual admissible-triple construction. It groups incoming and outgoing edges by middle vertex and generates all nontrivial transitions, excluding patterns that would start from an auxiliary exit or end at an auxiliary entry.

This helper is where the combinatorial topology surface is really formed.

`completeSwitchingCosts`

`completeSwitchingCosts[model, topology]` expands the switching-cost association over the full set of auxiliary triples, assigning zero to any admissible transition with no explicitly supplied cost. This makes downstream complementarity logic total rather than sparse.

4.4 Usage guidance

Users should think of this file as the package's gatekeeper. Any raw network or graph should enter the active MFGraphs workflow through `makeScenario`. Advanced users may inspect `scenarioData` and topology helpers, but bypassing the constructor usually means taking responsibility for normalization and validation manually.

References

1. Source file: `MFGraphs/scenarioTools.wl`
2. Workflow guidance: `CLAUDE.md`

Chapter 5

examples.wl

5.1 High-level explanation

`examples.wl` is the scenario-constructor and built-in example registry for the active MFGraphs package. It serves two related roles. First, it provides direct user-facing constructors such as `gridScenario`, `cycleScenario`, `graphScenario`, and `amScenario`. Second, it maintains a curated registry of named and numbered example factories accessed through `getExampleScenario`.

This module is therefore the most convenient bridge between the abstract scenario kernel and concrete test or benchmark workloads. It lets users create scenarios either from transparent topology arguments or from a standard benchmark catalog.

5.2 Low-level explanation of functions and functionality

gridScenario

`gridScenario[dims, entries, exits, sc, alpha, V, g]` constructs a scenario from a directed `GridGraph`. When `dims` is `{n}`, it creates a chain on vertices `1..n`; when `dims` is `{r, c}`, it creates an $r \times c$ grid with row-major integer labels.

Its implementation delegates to `scenarioFromGraph` and therefore inherits all validation and completion behavior from `makeScenario`. The main usage pattern is for synthetic benchmark or tutorial cases where the topology should be obvious from the constructor arguments.

cycleScenario

`cycleScenario[n, entries, exits, sc, alpha, V, g]` constructs a directed cycle $1 \to 2 \to \cdots \to n \to 1$. It is the simplest way to obtain cyclic benchmark scenarios without spelling out adjacency matrices manually.

Because it also routes through `scenarioFromGraph`, users get a normal typed scenario object immediately.

graphScenario

`graphScenario[graph, entries, exits, sc, alpha, V, g]` converts an arbitrary Wolfram directed `Graph` into a typed MFGraphs scenario. The key precondition is that the graph's vertex labels must remain compatible with the scenario kernel's integer-label policy.

This constructor is useful when topology is already represented as a Wolfram graph and the user wants to stay in that idiom.

amScenario

`amScenario[vl, am, entries, exits, sc, alpha, V, g]` constructs a scenario directly from a vertex list and adjacency matrix. It is the most explicit constructor in the file and is especially useful when benchmark cases are easier to define numerically than graphically.

This is also the constructor most often used by the internal example registry for fixed canonical cases.

getExampleScenario

`getExampleScenario` has two public forms. The one-argument form `getExampleScenario[n]` returns a six-argument factory `Function[{entries, exits, sc, alpha, V, g}, ...]`. The multi-argument convenience form resolves canonical switching costs when `sc = Automatic`, applies default Hamiltonian parameters, and immediately builds a scenario.

This design separates topology from boundary-condition choice. The registry stores topology-specific factories, while callers provide scenario-specific entries, exits, and optional overrides.

getExampleScenarioMetadata

`getExampleScenarioMetadata[key]` returns provenance and source metadata for a built-in example, when available. It returns `$Failed` for keys that are not in the registry. Use it when a script or paper-table builder needs to attach a human-readable origin to a scenario alongside its solution.

listExampleScenarios

`listExampleScenarios[]` returns the sorted list of all keys recognised by `getExampleScenario` — both the numeric legacy keys (`3`, `11`–`18`, `20`–`23`, `27`, `104`, `105`) retained for backward compatibility and the newer named keys (`"Jamaratv9"`, `"Grid0303"`, `"Camilli 2015 simple"`, ...). It is the canonical discovery API: tests, benchmarks, and example-coverage smoke runs iterate over its return value rather than hard-coding key lists.

scenarioFromGraph

`scenarioFromGraph[graph, entries, exits, sc, alpha, V, g]` is the private helper that packages graph-based inputs into the raw association form expected by `makeScenario`. It is the shared implementation core for `gridScenario`, `cycleScenario`, and `graphScenario`.

makeGridFactory

`makeGridFactory[dims]` returns a closure that behaves like `gridScenario[dims, ##, ...]&`. It exists so the built-in registry can store lightweight topology-specific factories rather than duplicating constructor code.

makeCycleFactory

`makeCycleFactory[n]` is the analogous closure constructor for cycle examples. It lets the registry represent a cycle topology as a reusable factory.

`makeAmFactory`

`makeAmFactory[vl, am]` is the adjacency-matrix version of the same idea. Most numbered benchmark cases rely on this helper because their topology is most clearly encoded as explicit matrices.

5.3 Registry data and constants

The file also defines several important constants. `$Y1In2OutAM`, `$Y2In1OutAM`, and `$Attraction4AM` are reusable adjacency patterns for benchmark families. `$CaseDefaultSC` stores canonical switching-cost sets keyed by example number or example name. Omitted Hamiltonian parameters use an `Automatic` sentinel in the example layer and are delegated to `makeScenario`, which is the canonical source for concrete Hamiltonian defaults.

The large `$ExampleScenarios` association is not itself a function, but it is central to the file's behavior. It maps numeric and string identifiers to factories for chains, Y-cases, attraction cases, Braess variants, Jamarat-style examples, paper examples, inconsistent-switching validation cases, and larger grids.

5.4 Usage guidance

Direct constructors are preferable when topology should be obvious from the call site. The example registry is preferable when reproducibility matters and the scenario should align with named benchmark cases already used elsewhere in the repository.

Representative usage patterns are:

```
s1 = gridScenario[{3}, {{1,10}}, {{3,0}}];
s2 = getExampleScenario[12, {{1,100}}, {{4,0}}];
s3 = getExampleScenario[8, {{1,80}}, {{3,0},{4,10}}];
```

References

1. Source file: `MFGraphs/examples.wl`
2. Scenario kernel dependency: `MFGraphs/scenarioTools.wl`

Chapter 6

unknownsTools.wl

6.1 High-level explanation

`unknownsTools.wl` is the layer that converts a completed scenario topology into a typed bundle of symbolic unknown variables. In the active MFGraphs architecture, this is the stage between scenario construction and system construction. It does not derive equations itself; instead, it creates the symbolic families that later modules place into those equations.

The file is deliberately narrow. Its purpose is to determine which `j`-variables, transition-flow variables, and `u`-variables should exist for a given scenario. This modularity is important because it keeps variable generation distinct from both topology generation and system assembly.

6.2 Low-level explanation of functions and functionality

`symbolicUnknowns`

`symbolicUnknowns[assoc]` is the typed head used to wrap the unknown bundle. It indicates that the wrapped association should be interpreted as a structured symbolic-variable record rather than an arbitrary association.

`symbolicUnknownsQ`

`symbolicUnknownsQ[x]` returns `True` exactly when `x` matches `symbolicUnknowns[_Association]`. It is the standard predicate used by downstream constructors that demand a typed unknown bundle.

`symbolicUnknownsData`

`symbolicUnknownsData[u]` returns the underlying association, while `symbolicUnknownsData[u,key]` performs lookup with `Missing["KeyAbsent", key]` fallback. This accessor is the canonical way to retrieve fields such as `"Js"`, `"Jts"`, and `"Us"`.

`makeSymbolicUnknowns`

`makeSymbolicUnknowns[s]` is the main public constructor. It accepts a typed scenario, a typed wrapper carrying an association, or a raw association. It first tries to reuse cached topology. If no topology is present, it attempts to rebuild auxiliary topology from the model. Once topology is available, it constructs the unknown bundle from the auxiliary pairs and triples.

This function is the standard entrypoint for the unknown-generation stage. In typical workflow code, users call:

```
unk = makeSymbolicUnknowns[s];
```

unknown

`unknown` is a reserved symbol for future numeric or grid-based solver fields. In the current exact symbolic graph-system phase, it is intentionally unused.

Its presence is historically and architecturally informative: the file leaves space for a broader solver boundary without confusing the current symbolic pipeline.

unknowns

`unknowns` is the reserved plural analogue of `unknown`. Like `unknown`, it is not active in the current exact symbolic graph-system constructors.

canonicalUPair

`canonicalUPair[{a,b}]` is a private helper that orients auxiliary boundary pairs consistently for `u`-variables. In particular, if the second endpoint is an auxiliary entry or auxiliary exit label, the pair is reversed.

This convention matters because it makes value-function variables stable and predictable at the boundary. Tests in the repository rely on this orientation rule.

makeSymbolicUnknownsFromPairsTriples

`makeSymbolicUnknownsFromPairsTriples[auxPairs, auxTriples]` is the private constructor that actually builds the typed unknown bundle. It populates five fields: `"Js"`, `"Jts"`, `"Us"`, `"AuxPairs"`, and `"AuxTriples"`.

The low-level mapping is straightforward but important. Two-point auxiliary pairs produce `j[a,b]` flow variables. Three-point auxiliary triples produce `j[r,i,w]` transition variables. Canonicalized pairs produce `u[a,b]` value variables.

6.3 Usage guidance

This module should be used through its main constructor rather than through manual symbolic-variable creation. The reason is not convenience alone. Topology-aware generation ensures that the unknown bundle is consistent with the scenario's auxiliary graph and transition structure.

The standard pattern is:

```
unk = makeSymbolicUnknowns[s];
js  = symbolicUnknownsData[unk, "Js"];
```

References

1. Source file: `MFGraphs/unknownsTools.wl`
2. Upstream dependency: `MFGraphs/scenarioTools.wl`

3. Downstream consumer: `MFGraphs/systemTools.wl`

Chapter 7

systemTools.wl

7.1 High-level explanation

`systemTools.wl` is the structural equation-system kernel of MFGraphs. It takes a typed scenario and a typed symbolic unknown bundle and builds a typed `mfgSystem` object containing the equations, inequalities, rules, and typed subrecords that represent a stationary mean field game on the network.

Architecturally, this module is where scenario semantics become algebra. It collects boundary conditions, flow balance, complementarity constraints, switching inequalities, and Hamiltonian relations into a consistent system object. The design is modular: four builders produce typed records for boundary, flow, complementarity, and Hamiltonian data, and those records are then merged into the final system association.

7.2 The mean field game system this module assembles

Fix a directed multigraph $G = (V, E)$ obtained by augmenting the user-supplied network with auxiliary entry and exit vertices. On each directed edge $(a, b) \in E$ the module introduces a flow variable $j[a, b]$ and a per-vertex value variable $u[a, b]$ representing the optimal cost-to-go evaluated at vertex a along the edge (a, b). Three structural blocks act on these unknowns.

Boundary conditions. For each entry vertex a with mass μ_a, the module emits the entry-current equation

$$\sum_{b\,:\,(a,b)\in E} j[a, b] \;=\; \mu_a,$$

and for each exit vertex a with terminal cost c_a the substitution rule $u[\cdot, a] \mapsto c_a$. These exit substitutions are propagated through the rest of the system at build time, so the symbolic solver never has to discover them.

Flow balance (Kirchhoff). At every interior original vertex $i \in V$, the splitting and gathering balances impose

$$j[\cdot, i] \;=\; \sum_{w} j[i,\, i,\, w] \qquad \text{(gathering)}, \qquad j[r,\, i,\, \cdot] \;=\; \sum_{w} j[r,\, i,\, w] \qquad \text{(splitting)},$$

where the auxiliary triples $\{(r, i, w)\}$ enumerate admissible transitions through i. These are the Kirchhoff laws that say "flow in equals flow out", lifted to the auxiliary topology so that switching costs can be charged on (r, i, w) rather than on i alone.

Hamiltonian residual (edge-level HJB). For every undirected edge $\{a, b\}$ the module records

$$u[a,b]-u[b,a]+j[a,b]-j[b,a] \;=\; \mathrm{nrhs}_{ab}, \qquad \mathrm{nrhs}_{ab} \;=\; \begin{cases} 0 & \text{if } \alpha = 1 \text{ (critical regime)}, \\ m - \mathrm{sign}(m)\, m^{\alpha} & \text{otherwise}, \end{cases}$$

which is the integrated form of the edge-level Hamilton–Jacobi–Bellman equation. The factor $\mathrm{sign}(m)$ makes the right-hand side orientation-aware so that one formula handles both travel directions on the undirected edge. The active critical-congestion solver path is exactly the case $\mathrm{nrhs}_{ab} = 0$.

Complementarity (LCP). Two families of disjunctive constraints encode the equilibrium condition that flow only exists on edges that are part of an optimal path. On every undirected physical edge $\{a, b\}$,

$$j[a,b] = 0 \;\;\vee\;\; j[b,a] = 0 \qquad \text{(alternative zero-flow)},$$

and on every admissible transition triple (r, i, w),

$$j[r,i,w] \;=\; 0 \;\;\vee\;\; u[w,i] + \mathrm{sc}(r,i,w) - u[r,i] \;=\; 0,$$

paired with the switching optimality inequality $u[w, i] + \mathrm{sc}(r, i, w) - u[r, i] \geq 0$. Together they form the linear complementarity problem (LCP) that the symbolic solvers in `solversTools.wl` reduce.

7.3 Low-level explanation of functions and functionality

`mfgSystem`

`mfgSystem[assoc]` is the typed wrapper for a structural system. It indicates that the wrapped association should be interpreted as a fully assembled equation-system object rather than a generic association.

`mfgBoundaryData`

`mfgBoundaryData[assoc]` wraps boundary-condition data such as entry equations, entry rules, and exit-value rules. It is the typed boundary subrecord stored inside a system.

`mfgFlowData`

`mfgFlowData[assoc]` wraps flow-balance and non-negativity data, including signed-flow expressions and balance equations derived from the auxiliary topology.

`mfgComplementarityData`

`mfgComplementarityData[assoc]` wraps switching-cost consistency results, complementarity alternatives, and optimality inequalities.

mfgHamiltonianData

`mfgHamiltonianData[assoc]` wraps the Hamiltonian residual structure, including left-hand and right-hand sides, placeholder cost-current variables, and the general Hamiltonian equation block.

mfgSystemQ, mfgBoundaryDataQ, mfgFlowDataQ, mfgComplementarityDataQ, mfgHamiltonianDataQ

These predicates recognize the corresponding typed wrappers. Their main role is defensive dispatch. They are used to keep constructors, accessors, and downstream logic attached to the right data shape.

makeSystem

`makeSystem[s]` is the main public constructor. In one-argument form it automatically builds the symbolic unknown bundle, then delegates to the two-argument form. In two-argument form, `makeSystem[s, unk]` builds all four typed subrecords and returns a complete `mfgSystem`.

This function is the canonical bridge from scenario semantics to symbolic constraints. Typical usage is:

```
sys = makeSystem[s];
```

systemData

`systemData[sys]` returns the underlying association. `systemData[sys,key]` first checks top-level keys and, if necessary, searches inside the typed subrecords before returning `Missing["KeyAbsent", key]`.

This makes it the standard read interface for every solver and plotting function that consumes system content.

systemDataFlatten

`systemDataFlatten[sys]` merges all top-level and typed-subrecord keys into a single flat association. It is primarily a compatibility and inspection helper.

Its low-level importance is that it preserves a bridge to older flat-system access patterns without abandoning the newer typed-record design.

getKirchhoffLinearSystem

`getKirchhoffLinearSystem[sys]` extracts the entry-current vector, Kirchhoff matrix, and corresponding variable order. It is a specialized linear-algebra helper for the structural system.

Users working directly on system analysis or certain solver extensions may use this function to access the linear core of the flow-balance structure.

getKirchhoffMatrix

`getKirchhoffMatrix[sys]` returns a similar data package but includes a third placeholder slot retained for backward compatibility. It is a compatibility-preserving interface over the same structural idea.

consistentSwitchingCosts

`consistentSwitchingCosts[sc][trip]` evaluates whether a particular switching-cost entry obeys a triangle-inequality-style consistency condition relative to alternative intermediate transitions.

This is a diagnostic or analytical helper rather than a standard end-user call. Its main importance is explanatory: it formalizes what the system means by switching-cost consistency.

isSwitchingCostConsistent

`isSwitchingCostConsistent[switchingCosts]` checks consistency over the whole switching-cost structure. It accepts both associations and list representations and returns either `True`, `False`, or symbolic consistency conditions.

This function directly influences how complementarity alternatives are constructed downstream.

altFlowOp

`altFlowOp[j][{a, b}]` builds the alternative

$$j[a, b] = 0 \ \vee \ j[b, a] = 0,$$

encoding the idea that, on an undirected physical edge, at most one orientation carries flow in the critical complementarity formulation. This is the building block of the disjunctive "alternative zero-flow" block in `ComplementarityData`.

flowSplitting

`flowSplitting[auxTriples][x]` selects triples that begin with the directed pair `x`. It is used in splitting-balance construction.

flowGathering

`flowGathering[auxTriples][x]` selects triples that end with the directed pair `x`. It is used in gathering-balance construction.

switchingCostLookup

`switchingCostLookup[sc]` returns a callable lookup function `f[r,i,w]` that retrieves the switching cost for a transition, defaulting to zero when absent. This isolates switching-cost representation details from the rest of the system builders.

ineqSwitch

`ineqSwitch[u, switching][r, i, w]` builds the local switching optimality inequality at junction i for the transition from r to w:

$$u[w, i] + \mathrm{sc}(r, i, w) - u[r, i] \ \geq \ 0.$$

It says that diverting through w cannot strictly improve on the value attached to r. When the switching cost is supplied as a function rather than a constant, it is applied to the transition flow $j[r, i, w]$ before the inequality is formed, which lets the system accommodate flow-dependent switching penalties.

altSwitch

`altSwitch[j, u, switching][r, i, w]` builds the complementarity alternative paired with that inequality:

$$j[r,i,w] = 0 \quad \vee \quad u[w,i] + \mathrm{sc}(r,i,w) - u[r,i] = 0.$$

Either the transition carries no flow, or the corresponding switching inequality is tight at i. The system collects one such alternative per admissible triple (r, i, w); the resulting disjunctive structure is the main source of branching for the symbolic solvers.

roundValues

`roundValues[x]` rounds numeric content to a standard precision. The file provides overloads for numbers, rules, lists, and associations. This is a small cleanup utility for making output more stable and readable.

exactBoundaryValue and exactBoundaryValues

These private helpers convert numeric boundary data into exact values, typically through `Rationalize[val, 0]`. They reject nonreal or nonnumeric boundary values by returning `Failure` objects. This exactification step matters because the active symbolic solver pipeline depends on exact structural equations wherever possible.

buildBoundaryData

`buildBoundaryData[s, topology]` constructs entry equations, entry rules, exit-value rules, and boundary metadata. It turns raw entry and exit values into exact symbolic constraints and substitutions suitable for the later system and solver pipeline.

buildFlowData

`buildFlowData[s, topology, unk]` constructs signed-flow expressions, non-negativity constraints for both `Js` and `Jts`, splitting-balance equations, gathering-balance equations, and associated rule forms.

This is one of the most structurally important builders because it transforms topology plus variable families into the conservation laws that define admissible flow behavior.

buildComplementarityData

`buildComplementarityData[s, topology, unk]` builds the switching-cost and complementarity layer. It checks switching-cost consistency, constructs alternative zero-flow constraints on undirected edges, builds transition-flow alternatives, assembles switching inequalities by vertex, and forms the optimality complementarity block.

This is the main source of logical disjunction in the final system.

boundaryPreservingAutomorphisms

`boundaryPreservingAutomorphisms[scenario]` returns the list of graph automorphisms of the scenario's underlying graph that also preserve the (vertex, value) multisets of `Entries` and `Exits`. It is the structural ingredient used to derive symbolic orbit equalities; the function returns the empty list when no nontrivial symmetry survives the boundary data.

`addSymmetryEqualities`

`addSymmetryEqualities[sys, scenario]` returns a new `mfgSystem` whose `EqGeneral` block has been extended with the orbit equalities

$$j[a,b] \;=\; j[\sigma(a),\sigma(b)], \qquad u[a,b] \;=\; u[\sigma(a),\sigma(b)],$$

for every boundary-preserving automorphism σ produced by `boundaryPreservingAutomorphisms`. The augmented system has the same solution set as `sys`, since any equilibrium is invariant under symmetries of both the graph and the boundary data. The benefit is structural: the solver enters the search with a smaller set of independent unknowns, often by a large factor on symmetric grids and cycles. The function is a no-op when the symmetry group is trivial.

`addOracleEqualities`

`addOracleEqualities[sys, oracleResult]` appends `j[e] == 0` to `EqGeneral` for every edge `e` that the oracle has classified as inactive. It consumes the association returned by `numericOracleClassify` (in the optional `numericOracle` subpackage); the function itself is symbolic. Before returning, it runs a `FindInstance` safety probe on the pruned system: if the over-pruned system has no feasible point, the original `sys` is returned unchanged. The function is a no-op when the oracle reports no inactive edges or did not converge.

`buildHamiltonianData`

`buildHamiltonianData[s, topology, flowData]` builds the Hamiltonian residual block `EqGeneral`. For every undirected edge $\{a,b\}$ it records

$$u[a,b] - u[b,a] + j[a,b] - j[b,a] \;=\; \mathrm{nrhs}_{ab}, \qquad \mathrm{nrhs}_{ab} \;=\; \begin{cases} 0 & \alpha_{ab} = 1, \\ m - \mathrm{sign}(m)\, m^{\alpha_{ab}} & \alpha_{ab} \neq 1, \end{cases}$$

where α_{ab} is the edge-specific exponent (`Alpha` globally, overridable per edge by `EdgeAlpha`). The equation is enforced unconditionally for every edge; in particular, zero-flow edges force $u[a,b] = u[b,a]$, which propagates through the switching inequalities and pins value variables at bypassed exit nodes to their terminal cost. Placeholder cost-current symbols are stored alongside `EqGeneral` for future non-critical/numeric work but do not participate in the critical solve.

7.4 Usage guidance

End users should normally interact with this module through `makeSystem` and `systemData`. More specialized helpers such as `getKirchhoffMatrix` or switching-consistency functions are most relevant when extending solvers, profiling structural growth, or debugging particular system blocks.

The canonical workflow is:

```
unk = makeSymbolicUnknowns[s];
sys = makeSystem[s, unk];
flowBlock = systemData[sys, "FlowData"];
```

References

1. Source file: `MFGraphs/systemTools.wl`
2. Upstream dependencies: `scenarioTools.wl`, `unknownsTools.wl`
3. Downstream consumer: `solversTools.wl`

Chapter 8

solversTools.wl

8.1 High-level explanation

`solversTools.wl` is the active symbolic solver layer for `mfgSystem` objects. All public solver entrypoints in this file work on the same basic principle: first preprocess the system structurally, then apply a final solving strategy suited to the residual logical form. The file therefore serves as both the solve-stage API and the shared solver-infrastructure layer.

A central architectural fact about this module is that the active public solvers only support *critical congestion* systems, meaning systems with `Alpha == 1` on every edge. This restriction is enforced explicitly by several public entrypoints. Another central fact is that the module distinguishes between exact linear preprocessing and the final residual-solve strategy. That is why multiple solver entrypoints can coexist while still sharing most of their setup logic.

8.2 Choosing a solver

The table below summarizes the public solver family. "Exact" means the solver returns either a rule list, a residual-equations record, or a no-solution signal — never a numeric approximation. "Backend" identifies whether the final residual solve uses a Wolfram primitive directly or a package-defined reducer on top of one.

Solver	Backend	When to use it
`reduceSystem`	`Reduce` (Reals)	Reference baseline. Conceptually simplest; can branch heavily on complementarity and time out on medium-sized systems.
`dnfReduceSystem`	DFS DNF reducer	*Default* for `solveScenario`. Equality-substitute-and-distribute, depth-first. Robust across the test suite.
`bfsDNFReduceSystem`	BFS DNF reducer	Breadth-first analog of the default. Statistically indistinguishable from DFS on the current benchmark suite (see `BENCHMARKS.md`); kept as a reference variant.

Solver	Backend	When to use it
`optimizedDNFReduceSystem`	Branch-state DNF	Use when the number of tracked unknowns is small enough that exact branch-state enumeration is cheaper than full DNF distribution. Falls back to `dnfReduceSystem` otherwise.
`activeSetReduceSystem`	Active-set + DNF	Specialized for the complementarity decomposition. Often the right pick on topologies with many switching alternatives; the oracle path (`solveScenarioWithOracle`) calls into this solver.
`booleanReduceSystem`	`BooleanConvert` then `Reduce`	DNF expansion at the Boolean layer with one `Reduce` per disjunct. Avoids some `Reduce` branching pathologies; controlled by `"DisjunctTimeout"` and `"ReturnAll"`.
`booleanMinimizeSystem`	`BooleanMinimize` then `Reduce`	Replaces `BooleanConvert` with minimal-DNF cover construction. Slower setup, smaller per-disjunct work; useful when the disjunction has many absorbable arms.
`booleanMinimizeReduceSystem`	Pruning + components + `BooleanMinimize`	Adds arm pruning and connected-component decomposition before `BooleanMinimize`. The most aggressive symbolic solver in the package; pays off on large systems whose complementarity graph decomposes.
`findInstanceSystem`	`FindInstance` (Reals)	When a single feasible rule set is sufficient and you do not need a full symbolic parameterization.
`directCriticalSystem`	Distance pruning + `FindInstance`	Restricted to systems with numeric zero switching costs and equal numeric exit values. Returns `Failure` otherwise. Fastest path when its preconditions hold.
`flowFirstCriticalSystem`	`LeastSquares` + `FindInstance`	Linear least-squares flow candidate followed by feasibility check. Useful when the flow block is over-determined linearly and the residual is small.

Rules of thumb.

- Start with `solveScenario[s]` (`dnfReduceSystem` underneath).
- If it times out on a large or highly symmetric system, try `solveScenarioWithOracle[s]` (opt-in `numericOracle` subpackage; uses `activeSetReduceSystem`).
- If you only need feasibility, use `findInstanceSystem`.

- If the system has the special structure required by `directCriticalSystem`, that solver is usually fastest.
- Use `reduceSystem` as a correctness baseline when developing new solvers.

8.3 Low-level explanation of functions and functionality

`reduceSystem`

`reduceSystem[sys]` is the most direct symbolic solver. It calls the shared preprocessing pipeline and then runs `Reduce[constraints, allVars, Reals]` on the result. If the system is fully determined, it returns a rule list. If the system remains underdetermined, it returns an association of the form `<|"Rules" -> rules, "Equations" -> residual|>`.

Its main advantage is conceptual simplicity. Its main weakness is that `Reduce` can time out or branch heavily on the complementarity structure.

`dnfReduce`

`dnfReduce[xp, sys]` is the module's private-style public reducer for disjunctive systems. It simplifies `xp && sys` by solving equalities, substituting solutions, and distributing across disjunctions. The three-argument form is used internally for one-conjunct processing.

This function is not the top-level solver most users call first, but it is the key algorithmic primitive behind `dnfReduceSystem`.

`dnfReduceSystem`

`dnfReduceSystem[sys]` uses the same preprocessing pipeline as `reduceSystem` but replaces the final `Reduce` call with `dnfReduce[True, constraints]`. This often helps on systems where explicit symbolic case splitting is preferable to a monolithic `Reduce`.

In the current repository phase, this is the default user-facing symbolic solver route.

`bfsDNFReduce` and `bfsDNFReduceSystem`

`bfsDNFReduce[xp, sys]` is the breadth-first analog of `dnfReduce`: at every depth it expands all surviving conjuncts one step before recursing, so that equality substitutions discovered on any sibling branch can be reused across the whole frontier through the shared `$dnfSolveCache`. `bfsDNFReduceSystem[sys]` is the system-level wrapper, parallel to `dnfReduceSystem` but using the BFS reducer. On the current benchmark suite the two strategies are statistically indistinguishable, so the depth-first variant remains the default; the BFS variant is shipped as a comparable reference.

`optimizedDNFReduceSystem`

`optimizedDNFReduceSystem[sys]` is an opt-in exact solver that uses a small-branch exact branch-state path when the number of tracked variables is sufficiently small, and otherwise falls back to the proven DNF reducer path.

Its role is to preserve exactness while improving performance on small residual branch families.

activeSetReduceSystem

`activeSetReduceSystem[sys]` is an opt-in exact active-set solver tailored to the critical-congestion complementarity structure. It splits the problem into deterministic constraints and active complementarity constraints, preprocesses the deterministic block, and then handles the active block either via branch-state methods or via DNF fallback.

This solver is structurally more specialized than `optimizedDNFReduceSystem`, and its value lies in exploiting the system's complementarity decomposition explicitly.

booleanReduceSystem

`booleanReduceSystem[sys, opts]` converts the preprocessed constraints to DNF via `BooleanConvert`, then calls `Reduce` independently on each disjunct. Since each disjunct is a pure conjunction, this can avoid some internal `Reduce` branching pathologies.

Its options are `"DisjunctTimeout"` and `"ReturnAll"`. The first bounds each disjunct-level `Reduce` call. The second chooses whether to return only the first parsed result or all non-false parsed results.

booleanMinimizeSystem and booleanMinimizeReduceSystem

`booleanMinimizeSystem[sys, opts]` replaces the `BooleanConvert` step of `booleanReduceSystem` with `BooleanMinimize[..., "DNF"]`, producing a minimal-DNF cover before each per-disjunct `Reduce`. `booleanMinimizeReduceSystem[sys, opts]` additionally prunes implausible arms and decomposes the residual constraints into independent components before invoking `BooleanMinimize` on each component separately. Both are exact and operate on the same critical-congestion preprocessing pipeline as the other solvers.

findInstanceSystem

`findInstanceSystem[sys, opts]` applies the shared preprocessing and then asks `FindInstance` for one feasible rule set on the remaining variables. It is useful when feasibility is more important than obtaining a complete symbolic parameterization.

Its main option is `"Timeout"`.

directCriticalSystem and flowFirstCriticalSystem

These are opt-in critical-congestion solvers preserved as proof-of-concept alternatives to the DNF and active-set families. `directCriticalSystem[sys]` is a graph-distance pruning heuristic that selects a candidate active set and then runs `FindInstance` on the residual; it is restricted to systems with numeric zero switching costs and equal numeric exit values, returning a `Failure` object otherwise. `flowFirstCriticalSystem[sys]` constructs a linear least-squares flow candidate via `LeastSquares` and then verifies feasibility with `FindInstance`. Neither solver is part of the default `solveScenario` path.

solutionReport and solutionBranchCostReport

`solutionReport[sys, sol]` returns an association summarizing a solver result: result kind, validation report, Kirchhoff residual, and transition-flow diagnostics. `solutionBranchCostReport[sys, sol]` ranks the branches of a residual disjunctive solution by total objective value without rewriting the solution itself. Both functions are diagnostics: they consume any solver output and produce

a stable inspection record, which makes them useful for benchmarking and for triaging unfinished branches.

`isValidSystemSolution`

`isValidSystemSolution[sys, sol]` checks whether a solver result satisfies the system blocks. It can return a simple Boolean or, with `"ReturnReport" -> True`, a detailed block-by-block validation report.

This function is critical operationally because MFGraphs solvers may return full rules, partial rules plus residual equations, or no-solution signals. Validation must therefore work across multiple result shapes.

`trackedVarQ`

`trackedVarQ[var]` identifies the variable heads that matter to solver post-processing. In the current active implementation, these are principally `j[__]` and `u[__]`. This keeps preprocessing and residual analysis focused on meaningful unknowns.

`mergeRules` and `normalizeRules`

`mergeRules` joins old and new rule sets while keeping the latest rule for a left-hand side. `normalizeRules` pushes the full rule set through the right-hand sides so accumulated substitutions become internally consistent.

These helpers are small, but they are vital for making multi-stage elimination results usable.

`topLevelEquations`

`topLevelEquations[constraints]` extracts top-level equalities from a conjunction. This is a structural helper used before linear elimination.

`extractLinearRules`

`extractLinearRules[equations, vars, existingRules]` solves eliminable linear equations for new substitution rules. It is the inner rule-extraction engine used during preprocessing.

`accumulateLinearRules`

`accumulateLinearRules[baseConstraints, vars, initRules]` repeatedly extracts linear rules from the system and returns both the reduced constraints and the accumulated rule set. This is one of the most important helper functions in the whole solver file because it performs the shared symbolic elimination that makes every final solver entrypoint lighter.

`attachAccumulatedRules`

`attachAccumulatedRules[result, rulesAcc]` merges preprocessing rules back into a final solver result. This preserves the user-facing contract that solutions should include all discovered substitutions, not only those from the final residual solve.

`branchToRules`, `branchStateReduceResult`, and related branch helpers

`branchToRules` extracts explicit variable rules from one branch. `branchStateReduceResult` reduces constraints by carrying surviving branches as exact rules plus residuals. Additional helpers such as `harvestDNFBranch` and branch-finalization logic perform the conversion from branch-state representations to user-facing rule or residual outputs.

These functions are the exact-branching machinery that distinguishes the optimized and active-set solvers from the simpler DNF path.

`buildSolverInputs`

`buildSolverInputs[sys]` is the shared preprocessing pipeline. It collects the relevant system blocks, applies exit-value substitutions, runs linear accumulation, and returns `{constraints, allVars, rulesAcc}`.

Conceptually, this function is the architectural heart of the solver file. The public solver family differs mainly in what happens *after* this helper returns.

`checkBlock`

`checkBlock[expr, tol]` evaluates whether a substituted constraint block is satisfied numerically within tolerance. It is one of the basic pieces behind `isValidSystemSolution`.

`solutionResultKind`

`solutionResultKind[sol]` classifies solver outputs into categories such as rule-based, branched, parametric, residual, no-solution, timeout, and related cases. This is especially useful in scripts, benchmarks, and diagnostics.

`dnfResidualDiagnostics`

`dnfResidualDiagnostics[sol]` provides lightweight information about residual DNF structure and tracked variables that remain unresolved. It is meant for analysis rather than direct solving.

`dnfReduceDiagnosticReport`

`dnfReduceDiagnosticReport[sys, opts]` runs the DNF reducer with instrumentation and returns a diagnostic association. It is a profiling and debugging helper rather than a normal production solver call.

8.4 Usage guidance

Typical users should begin with:

```
sol = dnfReduceSystem[sys];
validQ = isValidSystemSolution[sys, sol];
```

Advanced users can switch solver strategy by passing a different public solver into `solveScenario` or by calling the solver entrypoints directly on a prepared system. The main decision point is whether the residual problem is better handled by direct `Reduce`, DNF-style symbolic distribution, branch-state logic, or feasibility search.

References

1. Source file: `MFGraphs/solversTools.wl`
2. Upstream dependency: `MFGraphs/systemTools.wl`
3. Benchmark script context: `Scripts/BenchmarkSystemSolver.wls`

Chapter 9

orchestrationTools.wl

9.1 High-level explanation

`orchestrationTools.wl` is the high-level workflow glue for the active MFGraphs package. It does not introduce new mathematical data structures. Instead, it connects the existing scenario, unknown, system, and solver layers into convenient top-level calls.

This file is important because it presents the cleanest end-to-end public story of the current package. A user with a scenario object typically wants a solution, not a manual reconstruction of all intermediate stages. This module is what turns the staged architecture into a practical workflow.

9.2 Low-level explanation of functions and functionality

`solveScenario`

`solveScenario[s, solver]` is the main orchestration entrypoint. On a single scenario, it builds the symbolic unknown bundle with `makeSymbolicUnknowns`, builds the structural system with `makeSystem`, and then applies the selected solver. If no solver is supplied, it defaults to `dnfReduceSystem`.

The function also has a list overload: `solveScenario[{s1,s2,...}, solver]` maps the same workflow across multiple scenarios and returns a corresponding list of solutions.

Typical usage is:

```
sol = solveScenario[s];
sol2 = solveScenario[s, activeSetReduceSystem];
sols = solveScenario[{s1, s2}];
```

`SolveMFG`

`SolveMFG[assoc]` is a backward-compatibility wrapper for legacy raw-association input. It first calls `makeScenario` to convert the raw input into the current typed scenario representation. If conversion fails, it emits a message and returns the failure. Otherwise, it delegates to `solveScenario`. The two-argument form `SolveMFG[..., solver]` forwards the solver selection through to `solveScenario`.

Its main role is transitional. New code should generally prefer explicit `makeScenario` plus `solveScenario`, but `SolveMFG` preserves a familiar entrypoint for older calling styles.

Session-scoped memoization

`solveScenario` is memoized in a session-scoped cache (`$solveScenarioCache` in the private context) keyed by the hash of the scenario payload plus the solver symbol. Because every public solver in the package is a pure function of its `mfgSystem` input, this caching is safe: the first call on a given `(s, solver)` pair pays the full `makeSystem` plus solve cost, and subsequent calls return in essentially constant time. Direct calls into `dnfReduceSystem[sys]` or other low-level solvers bypass this cache by design — benchmarks that need cold timings should use those entrypoints directly.

`clearSolveCache`

`clearSolveCache[]` empties the session cache. It is the right call between benchmark passes when you want each pass to measure cold-start cost, and it can also be used to release memory in long-running interactive sessions. There is no return value worth keeping.

9.3 Usage guidance

For most end users, this file contains the friendliest solve API in the repository. If the caller already has a scenario, `solveScenario` is usually the right choice. If the caller is migrating older code based on raw associations, `SolveMFG` provides a compatibility bridge.

References

1. Source file: `MFGraphs/orchestrationTools.wl`
2. Upstream dependencies: `scenarioTools.wl`, `unknownsTools.wl`, `systemTools.wl`, `solversTools.wl`

Chapter 10

graphicsTools.wl

10.1 High-level explanation

`graphicsTools.wl` is the visualization layer of the active MFGraphs package. It exposes two plotters and one structural helper. `rawNetworkPlot` renders the physical network as the user sees it (real vertices and physical edges), and `richNetworkPlot` renders the *augmented state-space graph* (oriented-edge nodes, flow arcs, and transition arcs) that the system builders actually solve over. `augmentAuxiliaryGraph` produces the underlying augmented-graph data that `richNetworkPlot` consumes, so external tooling can build its own views without re-deriving the augmented structure.

The module deliberately ships only those two visual idioms. Every styling concern — whether boundary data is shown, which labels appear, how arcs are coloured, how anti-parallel edges are separated — is exposed through plot options rather than through additional public functions.

10.2 Public symbols

`rawNetworkPlot`

`rawNetworkPlot[s, sys, opts]` and `rawNetworkPlot[s, sys, sol, opts]` render the physical network. Real network vertices are drawn in gray; auxiliary entry/exit vertices and boundary edges are hidden by default. When a solution is supplied, shown vertex states are coloured by a u-value gradient (blue $\rightarrow$ red) while physical entry/exit vertices remain gray. Density labels can be overlaid for Tawaf-with-density systems.

Options (all defaults shown):

Option	Default	Effect
`ShowAuxiliaryVertices`	`False`	Show the auxiliary entry/exit vertices added by `makeScenario`.
`ShowBoundaryData`	`False`	Overlay boundary entry/exit data; forces `ShowAuxiliaryVertices` on.
`ShowBoundaryValues`	`True`	Show u-values on auxiliary exit vertices when boundary data is shown.
`ShowFlowLabels`	`Automatic`	Label edges with j-values; `True` when a solution is supplied.

Option	Default	Effect
`ShowValueLabels`	`Automatic`	Label vertex states with u-values; `True` when a solution is supplied.
`ShowDensityLabels`	`False`	Show inferred per-edge densities m (Tawaf-with-density).
`ColorFunction`	`Automatic`	Custom colour function over u-values (default: blue-to-red blend).
`ShowLegend`	`True`	Display the legend.
`GraphLayout`	`Automatic`	Forwarded to the underlying `Graph` call.
`PlotLabel`	`Automatic`	Title styling.
`ImageSize`	`Large`	Forwarded to the underlying `Graph` call.

`richNetworkPlot`

`richNetworkPlot[s, sys, opts]` and `richNetworkPlot[s, sys, sol, opts]` render the augmented state-space graph. Nodes are oriented-edge pairs $\{a, b\}$ representing the state "arriving at b along edge (a, b)". Edges come in two colours: flow arcs $j[a, b]$ in blue and transition arcs $j[r, i, w]$ in red. Arcs are drawn as quadratic Bezier curves so that anti-parallel pairs separate visually. Only nodes whose second component is an auxiliary entry/exit vertex receive boundary colouring.

Options (all defaults shown):

Option	Default	Effect
`ShowFlowEdges`	`True`	Include flow arcs $j[a, b]$. Set to `False` to obtain the transition-only graph.
`ShowBoundaryData`	`False`	Overlay entry-flow and exit-cost labels.
`ShowBoundaryValues`	`True`	Show u / exit-cost on boundary nodes.
`ShowFlowLabels`	`Automatic`	Label arcs with j-values; `True` when a solution is supplied.
`ShowValueLabels`	`Automatic`	Label nodes with u-values; `True` when a solution is supplied.
`UseColorFunction`	`False`	Colour nodes and eligible edges by u-values via `ColorFunction`.
`ColorFunction`	`Automatic`	Custom colour function over u-values.
`ShowLegend`	`True`	Display the legend.
`BendFactor`	`0.15`	Arc curvature as a fraction of edge length.
`GraphLayout`	`Automatic`	Forwarded to the underlying `Graph` call.
`PlotLabel`	`Automatic`	Title styling.
`ImageSize`	`Large`	Forwarded to the underlying `Graph` call.

`augmentAuxiliaryGraph`

`augmentAuxiliaryGraph[sys]` constructs the augmented state-space graph from the system's `AuxPairs` and `AuxTriples`. It returns an `Association` with keys:

- `"Graph"` — the Wolfram `Graph` object;
- `"Vertices"` — the list of oriented-edge node labels;
- `"FlowEdges"` — flow arcs corresponding to the $j[a, b]$ family;
- `"TransitionEdges"` — transition arcs corresponding to the $j[r, i, w]$ family;
- `"EdgeVariables"` — map from each arc to the symbolic variable it represents;
- `"EdgeKinds"` — map from each arc to either `"Flow"` or `"Transition"`.

This is the data `richNetworkPlot` renders. External tooling that wants to draw its own augmented-state view (e.g. in a notebook, in a publication figure, or in an animation) should consume this association directly rather than reimplementing the augmentation logic.

10.3 Usage guidance

A typical plotting workflow is:

```
s   = gridScenario[{3}, {{1, 10}}, {{3, 0}}];
sys = makeSystem[s];
sol = solveScenario[s];

raw  = rawNetworkPlot[s, sys, sol];                  (* physical view *)
rich = richNetworkPlot[s, sys, sol];                 (* augmented view *)
transOnly = richNetworkPlot[s, sys, sol, ShowFlowEdges -> False];
```

Choose `rawNetworkPlot` when the audience cares about the network as a road map or topology, and `richNetworkPlot` when they need to reason about transition structure and switching choices. The two plots are complementary: showing them side by side is often the clearest way to communicate what an MFGraphs solution means.

References

1. Source file: `MFGraphs/graphicsTools.wl`
2. Upstream dependencies: `scenarioTools.wl`, `systemTools.wl`
3. Test suite: `MFGraphs/Tests/graphicsTools.mt`

Part III

Opt-in subpackages

Chapter 11

Tawaf.wl (opt-in)

11.1 High-level explanation

`Tawaf.wl` is an *opt-in* subpackage that ships inside the `MFGraphs` directory but is intentionally *not* loaded by `Needs["MFGraphs‘"]`. It models the Tawaf ritual — counter-clockwise circumambulation of the Kaaba for a fixed number of rounds — by *unrolling* the circular ring into a long acyclic chain, then *coupling* the resulting per-round flow variables so that physically identical edges across rounds share the same congestion. Users opt in by loading the main package and then the subpackage:

```
Needs["MFGraphs‘"];
Needs["Tawaf‘"];
```

After loading, the subpackage's public symbols are available on the context path alongside the rest of the MFGraphs surface. The modelling notes that justify the unroll-then-couple construction live at `docs/research/notes/tawaf-model.md` in the repository.

11.2 Geometric construction

A scenario built with `makeTawafScenario[rounds, nodesPerRound, layers]` has rounds×nodesPerRound×layers logical vertices. The default value of `layers` is 1. Vertices are indexed by triples (r, p, l) with $1 \le r \le \text{rounds}$, $1 \le p \le \text{nodesPerRound}$, $1 \le l \le \text{layers}$, and flattened layer-major by

$$\texttt{tawafEncode}(r, p, l) \;=\; (l-1)\cdot \text{rounds}\cdot \text{nodesPerRound} \;+\; (r-1)\cdot \text{nodesPerRound} \;+\; p.$$

Edges come in two families:

- *Tangential* edges run forward only (counter-clockwise) within a fixed layer l: $(r, p, l) \to (r, p+1, l)$ when $p <$ nodesPerRound, and $(r, \text{nodesPerRound}, l) \to (r+1, 1, l)$ when $r <$ rounds. The very last position of the last round has no outgoing tangential edge.
- *Radial* edges, present only when `layers` > 1, run in both directions between adjacent layers at the same (r, p): $(r, p, l) \leftrightarrow (r, p, l+1)$.

Boundary data is generated automatically: entries of mass 100/layers are placed at $(1, 1, l)$ for each layer, and exits of terminal cost 0 are placed at $(\text{rounds}, \text{nodesPerRound}, l)$ for each layer. The Black Stone is modelled by setting $V = -5$ on every tangential edge leaving position $p = 1$ of any round; all other edges receive a baseline V (zero by default, a strictly negative `"BaselinePotential"` when the density extension is enabled). Construction metadata (`Rounds`, `NodesPerRound`, `Layers`, `Density`, `BaselinePotential`) is stored on the scenario at `scenarioData[s, "Tawaf"]`.

11.3 Cross-round coupling

The unrolled graph treats every logical edge as independent, but physically the same arc is traversed once per round. `makeTawafSystem` therefore groups the per-round flow variables by their *physical id*: tangential flows on $(p_a, p_b, l, \text{direction})$ across all rounds, and radial flows on $(p, l_a, l_b, \text{direction})$ across all rounds. Within each physical cohort the system rewrites `EqGeneral` and `AltOptCond` so that every logical flow is replaced by the cohort sum

$$j_{\text{logical}} \;\mapsto \sum_{\text{logical edges in the same cohort}} j_{\text{logical}},$$

ensuring that congestion acts on the shared physical flow rather than on each round in isolation. Singleton cohorts and the boundary group are left untouched.

11.4 Public symbols

`makeTawafScenario`

`makeTawafScenario[rounds, nodesPerRound]` and `makeTawafScenario[rounds, nodesPerRound, layers]` return a typed `scenario[...]` with the geometry described above. Options:

- `"Density" -> True` (default `False`) opts into the density-dependent cost extension and requires a strictly negative `"BaselinePotential"`.
- `"BaselinePotential" -> -1.0` (default) sets the negative baseline V that is added on top of the Black-Stone discount when `"Density"` is on.

The builder validates dimensions (`rounds` ≥ 1, `nodesPerRound` ≥ 2, `layers` ≥ 1) and returns a `Failure` object on violation.

`makeTawafSystem`

`makeTawafSystem[s]` and `makeTawafSystem[s, unk]` build an `mfgSystem`, then rewrite `EqGeneral` and `AltOptCond` with the cross-round coupling described above. When the scenario was built with `"Density" -> True`, the builder additionally:

- generates a per-physical-edge density family `m[{a, b}]`, keyed by the lex-smallest logical undirected edge in each cohort;
- appends a density–flow consistency block to the Hamiltonian record:

  $$j_{\text{phys}}^2 \;+\; 2\,V\,m[\{a,b\}] \;+\; 1 \;=\; 0,$$

  where j_{phys} is the signed physical flow on the cohort and V is the per-edge potential;
- appends the positivity inequalities `m[{a, b}] > 0`.

If the scenario does not carry the `Tawaf` metadata block, the function returns a `Failure`.

`tawafEncode`

`tawafEncode[r, p, l, rounds, nodesPerRound]` returns the layer-major flat vertex index for (r, p, l). It is the single source of truth for the codec; downstream code that needs to address Tawaf vertices by coordinate should call this helper rather than re-deriving the formula. (The inverse decoder is private to the subpackage.)

`m`

`m[{a, b}]` is the per-physical-edge density symbol used by the density extension. One symbol is generated per non-boundary physical undirected edge, keyed by the lex-smallest logical undirected edge in the cohort. `EqDensityFlow` contains exactly one consistency equation per such `m`.

`tawafDensities`

`tawafDensities[sys, sol]` derives the per-physical-edge densities from a solution `sol` of the (j, u) part of a Tawaf-with-density system. For each $m[\{a, b\}]$, it locates the consistency equation in which it appears, substitutes the rules from `sol`, and solves the resulting linear equation for $m[\{a, b\}]$. The return value is an `Association` of rules $\{m[\{a, b\}] \to \text{value}\}$; it is empty when the system was not built with `"Density" -> True`. The result is designed to be `Join`-ed with `sol` so that downstream code sees a complete (j, u, m) rule list.

11.5 Usage guidance

A typical density-extended workflow is:

```
Needs["MFGraphs`"];
Needs["Tawaf`"];

s   = makeTawafScenario[3, 6, "Density" -> True];
sys = makeTawafSystem[s];
sol = solveScenario[s];
mRules = tawafDensities[sys, sol];
full   = Join[sol, Normal[mRules]];
```

For scenarios that are not circumambulations, use the general scenario constructors (`makeScenario`, `gridScenario`, `cycleScenario`, etc.) instead. Tawaf is a specialized builder, not a general-purpose backend.

References

1. Source file: `MFGraphs/Tawaf.wl`
2. Test suite: `MFGraphs/Tests/tawaf.mt`
3. Modelling notes: `docs/research/notes/tawaf-model.md`
4. Loader status: opt-in (not in `MFGraphs.wl`'s `BeginPackage` dependency list).

Chapter 12

numericOracle.wl (opt-in)

12.1 High-level explanation

`numericOracle.wl` is an *opt-in* subpackage that pairs a fast numeric classifier with the symbolic critical-congestion solver. The classifier runs a Linear Programming (LP) relaxation of the complementarity system, reads off which flow variables look active, inactive, or ambiguous at a feasible vertex, and returns that classification as a symbolic-only association. Downstream, the symbolic bridge `addOracleEqualities` (declared in `systemTools.wl`) consumes that association to append `j[e] == 0` equalities for the edges the oracle marks inactive, pruning the symbolic system before a solver runs.

This is the only file in the package that intentionally introduces floating-point work. Floats are confined to the LP relaxation; everything that crosses back into the symbolic layer carries only variable names and `True`/`False` classifications. The subpackage is loaded explicitly:

```
Needs["MFGraphs`"];
Needs["numericOracle`"];
```

12.2 Low-level explanation of functions and functionality

`numericOracleClassify`

`numericOracleClassify[sys]` drops the complementarity disjunctions from `sys`, calls `FindInstance` over the resulting LP relaxation, and classifies each flow variable from one feasible point. Variables whose value is within `"TightThreshold"` of zero are reported as `Inactive`; variables larger than `"SafeThreshold"` are reported as `Active`; everything in between is `Ambiguous`. The return value is an association `<|"Inactive" -> {...}, "Active" -> {...}, "Ambiguous" -> {...}, "Converged" -> True|False|>`.

The options `"TightThreshold"` (default 10^{-8}), `"SafeThreshold"` (default 10^{-3}), and `"Timeout"` (default 30s) control the LP and its interpretation.

`numericFictitiousPlayClassify`

`numericFictitiousPlayClassify[sys, opts]` is an iterative refinement of the basic classifier inspired by the archived fictitious-play backend. Each iteration solves the LP relaxation, classifies edges by current flow, and tightens the next iteration's constraints by enforcing complementarity at confidently classified edges. The loop terminates when the support classification is stable for

`StableIterationsLimit` consecutive iterations, or when `MaxIterations` is exhausted. The return shape matches `numericOracleClassify`, so the same `addOracleEqualities` bridge consumes it.

`solveScenarioWithOracle`

`solveScenarioWithOracle[s]` is the high-level wrapper for the oracle pipeline. It composes:

```
unk    = makeSymbolicUnknowns[s]
sys    = makeSystem[s, unk]
sym    = addSymmetryEqualities[sys, s]
oracle = numericOracleClassify[sym]
pruned = addOracleEqualities[sym, oracle]
result = activeSetReduceSystem[pruned]
```

If `result` carries `"Residual" -> False` (the over-pruned system has no feasible point), the wrapper transparently falls back to `activeSetReduceSystem[sym]` on the symmetry-augmented but unpruned system and returns that result. `solveScenarioWithOracle[s, solver]` uses the supplied solver in place of `activeSetReduceSystem`. Options `"TightThreshold"`, `"SafeThreshold"`, and `"Timeout"` are forwarded to `numericOracleClassify`.

12.3 Usage guidance

The oracle path is most useful on systems with many edges whose support pattern is hard for a pure symbolic solver to discover by case analysis but easy for an LP to recover. For small or generic systems, the unpruned symbolic solvers are typically faster and avoid the LP setup cost entirely.

A typical call is:

```
Needs["MFGraphs`"];
Needs["numericOracle`"];

s   = getExampleScenario[21, {{1, 100}}, {{4, 0}}];
sol = solveScenarioWithOracle[s];
```

References

1. Source file: `MFGraphs/numericOracle.wl`
2. Symbolic bridges: `addOracleEqualities`, `addSymmetryEqualities` (declared in `systemTools.wl`).
3. Test suite: `MFGraphs/Tests/numeric-oracle.mt`.
4. Loader status: opt-in (not in `MFGraphs.wl`'s `BeginPackage` dependency list).